A Fused Carry-Save Processing Element with Bias-Balanced Approximation for Efficient Systolic Matrix Multiplication

Anh Q. Vu, Dung T. Le
FPT University, Ho Chi Minh Campus, Ho Chi Minh City, Vietnam
anhvuvlsi@gmail.com, dunglt96@fe.edu.vn

***Abstract*—Systolic arrays accelerate the matrix multiplication that dominates deep neural network (DNN) inference, and fusing partial-product generation with accumulation lowers the cost of each processing element (PE). A recent fused design based on positive and negative partial-product cells (PPC/NPPC) reports large energy savings but leaves two issues that matter for long DNN accumulations. First, its running sum is stored in resolved binary form, so a carry-propagating ripple must complete inside every clock cycle and the accumulator cannot be widened to avoid overflow without lengthening the critical path. Second, its approximate cells introduce a *systematic* (biased) error whose magnitude grows linearly with the dot-product length. This paper proposes a fused carry-save PE that keeps the same PPC/NPPC cells but stores the accumulator redundantly as two words merged only once, at drain time; we prove an integer Baugh-Wooley identity that reduces the entire signed-arithmetic correction to a single boundary constant, and we show that the low half of the carry word is structurally zero. We further characterize the approximate-cell bias, which we find to be opposite in sign to its truth-table mean, and cancel it with a complementary cell interleaved by row parity and a free drain-time constant. On a bit-accurate model, RTL, and open-source synthesis (technology-independent and Xilinx xc7z020), the fused PE reduces the cycle-limiting logic depth from 47 to 26 gate levels (16 to 12 LUT levels) at a 4 to 7% cell increase, with the depth independent of accumulator width; because the cells are unchanged, the baseline's per-cell energy savings are retained. For a length-1152 layer the biased design shifts 54.7% of activations by more than 2 LSB, which our balanced, constant-compensated design reduces to 0.8% while preserving int8 accuracy. All functional claims are verified exhaustively.**

***Index Terms*—Systolic array, matrix multiplication, approximate computing, carry-save accumulation, processing element, Baugh-Wooley multiplier, DNN accelerator, error bias.**

I. Introduction

DEEP neural networks (DNNs) rest on large-scale matrix multiplication, and both training and inference are dominated by multiply-accumulate (MAC) operations [2], [3]. Systolic arrays are a favoured substrate for this kernel because a regular grid of processing elements (PEs) sustains high throughput with local communication [4], [5]. Since the array replicates one PE many times, the area, delay, and energy of that PE set the efficiency of the whole accelerator, and approximate computing is widely used to trade a bounded loss of accuracy for lower PE cost [6]–[8].

A recent design by Jaswal et al. [1] advances this line by fusing multiplication and accumulation inside the array: it builds each PE from positive and negative partial-product cells (PPC/NPPC) that compute $a \times b + c$ directly, supports signed operation through the Baugh-Wooley scheme [12], and offers an approximate variant controlled by a factor k. It reports up to 16% and 68% energy savings for exact and approximate 8×8 arrays. We take that design as our baseline and observe two limitations that become important for the long dot products of convolutional layers, where the contraction length is $K = 9C_{\text{in}}$ (typically 576 to 4608 after im2col):

- Resolved accumulator: the running sum is kept in resolved (non-redundant) binary form, so adding each product requires a carry-propagating adder, the ripple "accumulation stage" at the foot of the array, whose serial tail must complete every cycle. Widening this accumulator to prevent overflow lengthens the tail, coupling numerical range to clock period.
- Biased approximation: the approximate cells inject an error that is systematic rather than zero-mean. Over a length-K dot product a per-MAC bias μ accumulates as μK (linear), whereas zero-mean noise would grow only as $\sqrt{K}$; for DNN-scale K this bias, not the variance, dominates the error budget and corrupts downstream requantization.

This paper keeps the baseline's PPC/NPPC cells, so its per-cell energy characterization carries over unchanged, and addresses both limitations at the level of how the accumulator is stored and corrected. Our contributions are: (i) a fused carry-save PE that stores the accumulator as two redundant words and merges them only once, at drain time, removing the per-cycle ripple and making the critical-path depth independent of accumulator width; (ii) an integer Baugh-Wooley identity (Theorem 1) that folds the entire signed-arithmetic correction into a single configuration-time constant, plus a structural-zero lemma (Lemma 1) that halves the carry word's storage; and (iii) a bias-balanced approximate cell family that we characterize in-circuit and correct with a free drain-time constant, converting the linear error growth into a $\sqrt{K}$ floor. All functional claims are verified exhaustively, and timing/area are reported from technology-independent and FPGA synthesis.

The rest of the paper follows the organization of the baseline. Section II reviews systolic matrix multiplication and the fused PPC/NPPC PE we build on. Section III presents the

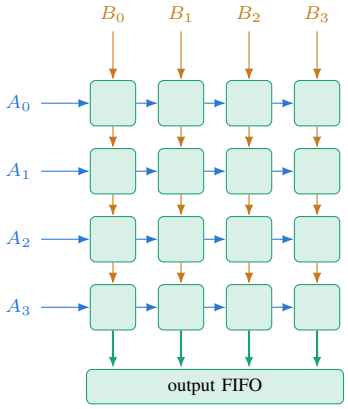


Fig. 1. 4×4 output-stationary array. Rows of A (blue) flow right, columns of B (amber) flow down; each C_{ij} accumulates in place and drains downward (teal). The proposed PE is a drop-in replacement for the baseline PE.

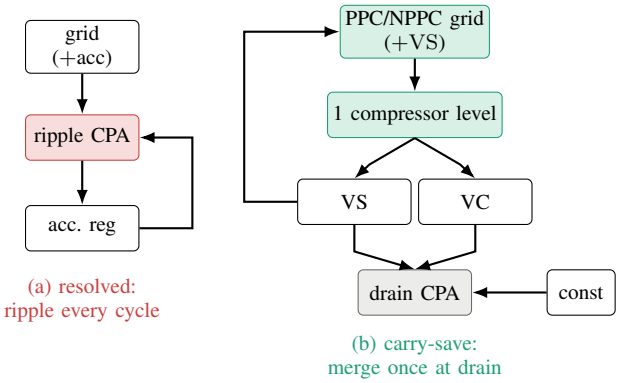


Fig. 2. PE datapath. (a) The resolved baseline places a carry-propagating adder inside the accumulation loop. (b) The proposed fused carry-save PE injects VS into the grid, compresses once with carries parked in VC, and merges VS + VC only at drain, where the sign/compensation constant is added.

proposed carry-save PE and its proofs. Section IV describes the hardware implementation and functional verification. Section V reports results, Section VI evaluates DNN and image-processing applications, and Section VII concludes.

II. Systolic-Array Matrix Multiplication and the Baseline PE

A. Output-Stationary Array

An $N \times N$ output-stationary array computes $C = AB$ with one PE per output C_{ij} (Fig. 1). Rows of A enter from the left and advance one cell per cycle; columns of B enter from the top and advance downward. With row i and column j skewed by i and j cycles, operands A_{ik} and B_{kj} meet in $\mathrm{PE}(i,j)$ at cycle $t = i+j+k$, so each PE accumulates its dot product in place and the last result is available after $3N-2$ cycles [4], [17]. The partial sum never leaves a PE during computation; only finished results are drained.

B. Fused PPC/NPPC Processing Element

The baseline PE [1] replaces the conventional "multiplier-then-adder" with a single grid of bit-level cells that folds $+c$ into the multiplication. An exact PPC forms the partial product $p = a_i b_j$ and adds it to the incoming sum and carry, i.e. it realizes a full adder $(S_{\text{out}}, C_{\text{out}}) = \mathrm{FA}(p, C_{\text{in}}, S_{\text{in}})$; an NPPC is identical with p replaced by $\overline{p}$, providing the inverted partial products required by two's-complement (Baugh-Wooley) multiplication [12]. For an N-bit PE the grid holds N^2 cells, of which $2N-2$ are NPPC, placed exactly where one operand index equals the sign position $N-1$, and the remainder PPC. The running partial sum enters the grid through its otherwise-unused sum/carry inputs, and a ripple carry-propagate stage at the foot of the array resolves the final value each cycle. The array latency remains $3N-2$ cycles; 4-bit and 8-bit signed PEs are obtained by tiling the same cells.

C. Approximate Cells

The approximate PPC discards the exact carry network. From the baseline's Table I, its behaviour is $C_{\text{out}} = p$ and $S_{\text{out}} = (S_{\text{in}} + C_{\text{in}}) \cdot \overline{p}$, replacing a full adder's XOR-and-majority logic with two gates; the approximate NPPC is its complement, $C_{\text{out}} = (S_{\text{in}} + C_{\text{in}}) \cdot \overline{p}$ and $S_{\text{out}} = \overline{C_{\text{out}}}$. Each errs on 5 of 16 input rows, always by ± 1 (error rate $5/16$). Approximation is confined to cells of weight below k. The quality of such inexact arithmetic is usually summarized by the mean and normalized error distance (MED, NED) [13]; as Section III-C shows, for accumulating structures the sign of the mean error is what governs end-to-end accuracy, because it is what accumulates coherently over a dot product.

III. Proposed Carry-Save Processing Element

A. Redundant Accumulator

We store the running total as two registers VS and VC under the single invariant

$$\mathrm{VS} + \mathrm{VC} \equiv \textstyle\sum_t a_t b_t \pmod{2^W}, \tag{1}$$

where W is the accumulator width. Any pair satisfying (1) is admissible. Each cycle, the grid's leftover sum/carry bits and the previous VC are combined by a single level of compressors (the same full/half adders as the cells), and every carry produced is written to a VC slot one weight higher, to be consumed next cycle. Because registers do not ripple, the only end-to-end carry propagation is the final merge, performed once per K cycles by a drain carry-propagate adder (CPA) that is shared by an entire array column and exercised only at read-out. Fig. 2 contrasts this datapath with the resolved baseline: the expensive addition moves out of the per-cycle loop and to the array boundary.

B. Constant-Folded Signed Correction

Theorem 1 (Integer Baugh-Wooley identity)**.** For N-bit operands, the signed product equals the non-negative sum of the grid's AND/NAND partial products plus a fixed integer offset:

$$P_{\text{signed}} = P_{\text{raw}} + 2^N - 2^{2N-1}. \tag{2}$$

Algorithm 1: carry-save accumulate and drain

Input: stream $\{(a_t, b_t)\}_{t=1}^{K}$; mode $\in$ {exact, approx}; sign s
Init: $\mathrm{VS} \leftarrow s\,K(2^N - 2^{2N-1}) + \mathrm{comp}$; $\mathrm{VC} \leftarrow 0$
for $t = 1$ **to** K:
  $G \leftarrow \mathrm{grid}(a_t, b_t, \mathrm{VS}; \mathrm{mode}, s)$ // fused $ab + \mathrm{VS}$
  $(\mathrm{VS}, \mathrm{VC}) \leftarrow \mathrm{compress}(G, \mathrm{VC})$ // carries $\rightarrow$ VC, no ripple
Drain: $C \leftarrow \mathrm{CPA}(\mathrm{VS} + \mathrm{VC})$; **return** C

Fig. 3. One fused carry-save PE. The invariant (1) holds after every iteration; the only carry-propagating addition is the drain.

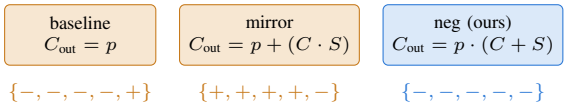


Fig. 4. Approximate cell family with the error-sign signature over the five erroneous rows. The three cells share the sum path and error rate; only the dominant error's direction differs.

Proof sketch. Expanding the two's-complement product and applying Baugh-Wooley sign extension replaces each negatively weighted cross term by its complement plus a correction. Collecting the constant terms of that substitution over all bit positions yields the width-independent offset $2^N - 2^{2N-1}$, an identity over $\mathbb{Z}$ rather than modulo 2^W. □

Because (2) holds over the integers, summing over K products gives $\sum P_{\text{signed}} = \sum P_{\text{raw}} + K\left(2^N - 2^{2N-1}\right)$; the entire signed correction is therefore a single constant, loaded into VS at clear (or added at drain). The baseline's per-cycle sign-handling hardware, namely a forced carry-in, an inverter, and an XNOR, is removed.

Lemma 1 (Structural zeros)**.** $\mathrm{VC}[0..N]$ is identically zero and need not be implemented.

Proof. Weights $0..N-1$ complete inside the grid: each produces one output bit and every carry it generates is absorbed by the next row's carry input. The first weight at which a leftover carry can appear is N, whose carry is parked at weight $N+1$. Hence no write to VC occurs at or below weight N. □

Lemma 1 reduces the redundant accumulator's flip-flop overhead from a naive $2\times$ to about $1.7\times$, which the two flip-flops per LUT of the target fabric absorb. Algorithm 1 (Fig. 3) summarizes one PE.

C. Bias-Balanced Approximate Cells

All approximate cells share the two-gate sum path and differ only in the carry gate (Fig. 4): the baseline cell [1] uses $C_{\text{out}} = p$; a mirror cell uses $C_{\text{out}} = p + (C_{\text{in}} \cdot S_{\text{in}})$; and the proposed neg cell uses $C_{\text{out}} = p \cdot (C_{\text{in}} + S_{\text{in}})$, reusing the OR already computed for the sum. Each errs on 5 of 16 rows by ± 1; the members differ only in the sign pattern of those five errors.

The key observation is that the truth-table mean mispredicts the in-circuit bias. The baseline cell errs low on four rows and high on one, yet that single high row, a fresh partial product landing on empty sum/carry inputs, is the most frequent event in the low-weight columns, so the measured bias is positive: $+7.78$ LSB per MAC for uniform operands and $+3.42$ for

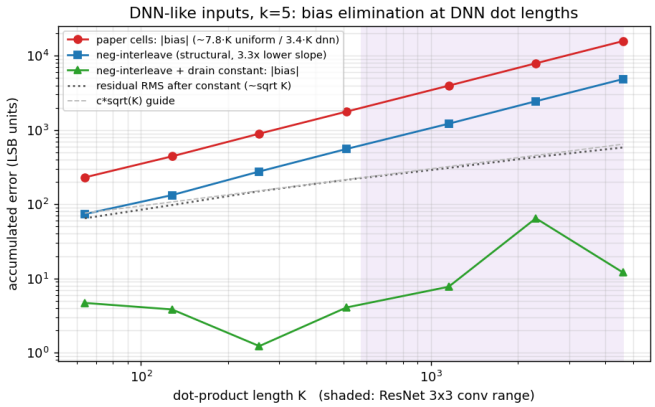


Fig. 5. Measured accumulated error versus dot-product length K (log-log, DNN-like operands, k=5). The baseline's biased cells grow linearly ($|\text{bias}| \approx 7.8K$ uniform, $3.4K$ DNN); the neg-interleave lowers the slope $3.3\times$; adding the free drain-time constant collapses the error to a $\sqrt{K}$ floor (dotted guide). Shaded: the K range of 3×3 ResNet convolutions.

DNN-like operands (Table II). The mirror cell shares that dominant row and is therefore worse. The neg cell instead errs -1 on the dominant row; interleaving baseline and neg cells by row parity within the approximate region lets the opposite-signed errors cancel, cutting the bias to $+2.84/+1.05$ LSB per MAC, a $3.3\times$ reduction, while preserving the error rate. Because heavier neg weightings overshoot negative, the family brackets zero, so bias becomes a tunable design knob.

Lemma 2 (No approximate NPPC)**.** For $k \leq N-1$ the approximate region contains no NPPC cell.

Proof. Every NPPC has one index equal to $N-1$, so its weight is at least $N-1$; the approximate region requires weight $< k \leq N-1$. The two are mutually exclusive. □

By Lemma 2 the baseline's approximate NPPC is never instantiated and is removed. Finally, the residual bias is empirically a per-MAC constant (flat across $K \in [64, 4608]$ to within 2%), so a single calibrated constant $-\lfloor \hat{\mu} K \rceil$ cancels it; it is added by the drain CPA that already exists (Fig. 2), at no per-cycle cost. Fig. 5 shows the effect.

IV. Hardware Implementation and Verification

The PE is described in Verilog as a registered datapath: input registers for a and b, the PPC/NPPC grid, one compressor level writing VS and VC, and a shared drain CPA. A clear signal loads the folded constant of Theorem 1 into VS. The PE composes into the $N \times N$ output-stationary array of Fig. 1 without any change to the dataflow; only what a PE loads at reset and reads at drain differs from the baseline, so array-level control, skewing, and latency $(3N-2)$ are unchanged.

Correctness is established at two levels against a bit-accurate golden model that implements the cells, the carry-save accumulation, and the constant folding. The cycle invariant (1) was checked exhaustively over all 65,536 operand pairs at two accumulator widths for both signed and unsigned modes

TABLE I
SYNTHESIS COMPARISON (PER PE). GATE DEPTH IS THE LONGEST TOPOLOGICAL PATH; LUT FIGURES TARGET XC7Z020.

Design	Gate depth	LUT depth	Cells
Baseline PE, exact [1]	47	16	534
Baseline PE, approx (k=5)	43	19	475
Carry-save PE, exact (28 b)	**26**	**12**	557
Carry-save PE, approx (28 b)	**26**	14	507
Carry-save PE, approx (17 b)	**26**	n/a	n/a

TABLE II
APPROXIMATE-CELL BIAS PER MAC (LSB).

Cell configuration	Uniform	DNN-like
Baseline approx [1]	$+7.78$	$+3.42$
Mirror	$+9.18$	$+4.20$
Neg-interleave (ours)	$+2.84$	$+1.05$
Neg-interleave + drain const (ours)	≈ 0	≈ 0

(1,150,976 checks, zero mismatches). The RTL matched the golden model on all replayed vectors across ten configurations, and 48 accumulate-and-drain runs up to $K = 1152$ were exact. A 4×4 system built from 16 PEs reproduced the integer product exactly.

V. RESULTS AND DISCUSSION

Synthesis uses Yosys with a technology-independent flow (longest topological path via `ltp`) for gate-level depth and cell counts, and `synth_xilinx` targeting the xc7z020 (PYNQ-Z2) for LUT-level depth and utilization. We do not re-run the baseline's 90-nm ASIC flow; because the cells are functionally identical, its per-cell energy characterization carries over, and our results isolate the separate timing and accuracy effects of the carry-save accumulator and the bias correction.

A. Timing and Area

Table I reports synthesis. The fused carry-save PE reduces the cycle-limiting depth from 47 to 26 gate levels (16 to 12 LUT levels), a 40 to 45% reduction, at a 4 to 7% increase in cell count. The depth is width-invariant: 26 levels at both 17- and 28-bit accumulators, whereas the resolved baseline lengthens with width. The shorter, width-decoupled path is what lets the clock period stay fixed as the accumulator grows wider.

B. Accumulation Error

Table II lists per-MAC bias. Table III evaluates a length-1152 layer with DNN operands: a 17-bit resolved accumulator wraps on 72.4% of dot products with uniform operands, while the carry-save accumulator, widened at no depth cost, does not. The baseline approximate cells shift activations by $+4.33$ LSB on average, moving 54.7% of activations by more than 2 LSB and flipping 8.5% of ReLU decisions; the balanced, constant-compensated design reduces these to $+0.02$ LSB, 0.0%, and 0.8%, and cuts the total RMS error by $22\times$.

TABLE III
LENGTH-1152 LAYER: ACTIVATION ERROR (DNN OPERANDS).

Design	mean err.	>2 LSB	ReLU flips
Baseline approx (k=5)	$+4.33$	54.7%	8.5%
Balanced + const (ours)	$\mathbf{+0.02}$	**0.0%**	**0.8%**

C. Discussion

Two findings generalize. First, for accumulating arithmetic the truth-table mean mispredicts the in-circuit bias, because input rows are not equiprobable; cell selection should be driven by in-circuit statistics. Second, bias, not variance, is the main limit at DNN scale, since it grows linearly in K; a sign-tunable cell family plus a boundary constant converts that into a $\sqrt{K}$ floor. The compensation constant is calibrated on a small sample and assumes an operand distribution; under distribution shift the structural interleave still limits the residual (its bias is $2.4\times$ smaller than the baseline's when the constant is miscalibrated). The benefit of approximation is modest at very small K, where no statistical cancellation occurs; the target regime is the long dot products of convolutional layers.

VI. APPLICATION TO DNN AND IMAGE PROCESSING

We evaluate the PE on the applications used to motivate it, comparing against the baseline. All arithmetic runs through the verified PEs.

A. Quantized DNN Inference

An int8 multilayer perceptron ($64 \to 128 \to 10$) on the digits dataset, quantized following common integer-only practice [14], is executed with every MAC performed by the PEs (Fig. 7). The floating-point reference reaches 96.00%. A resolved 17-bit PE loses accuracy to overflow (94.89%), whereas the carry-save PE in exact and balanced-approximate modes preserves floating-point accuracy (96.00%); adding the compensation constant yields 95.78%. This confirms that the width-decoupled accumulator removes the overflow penalty and that the balanced approximation is accuracy-neutral at this scale.

B. DCT and Edge Detection

The array performs an 8×8 integer-scaled DCT [16] for image compression and a 3×3 Laplacian convolution for edge detection, with image quality measured by SSIM [15] against the exact design. Fig. 6 shows the carry-save reconstructions: the exact PE is bit-identical to true arithmetic, and the balanced-approximate PE degrades gracefully with the approximation factor k (validated range $k \leq N - 1$, by Lemma 2). Table IV compares our balanced cells with the baseline's biased cells under an otherwise identical dataflow: the balanced design attains equal or higher SSIM at every k on both applications, confirming that removing the accumulation bias improves output quality while retaining the approximate cells' hardware benefit.

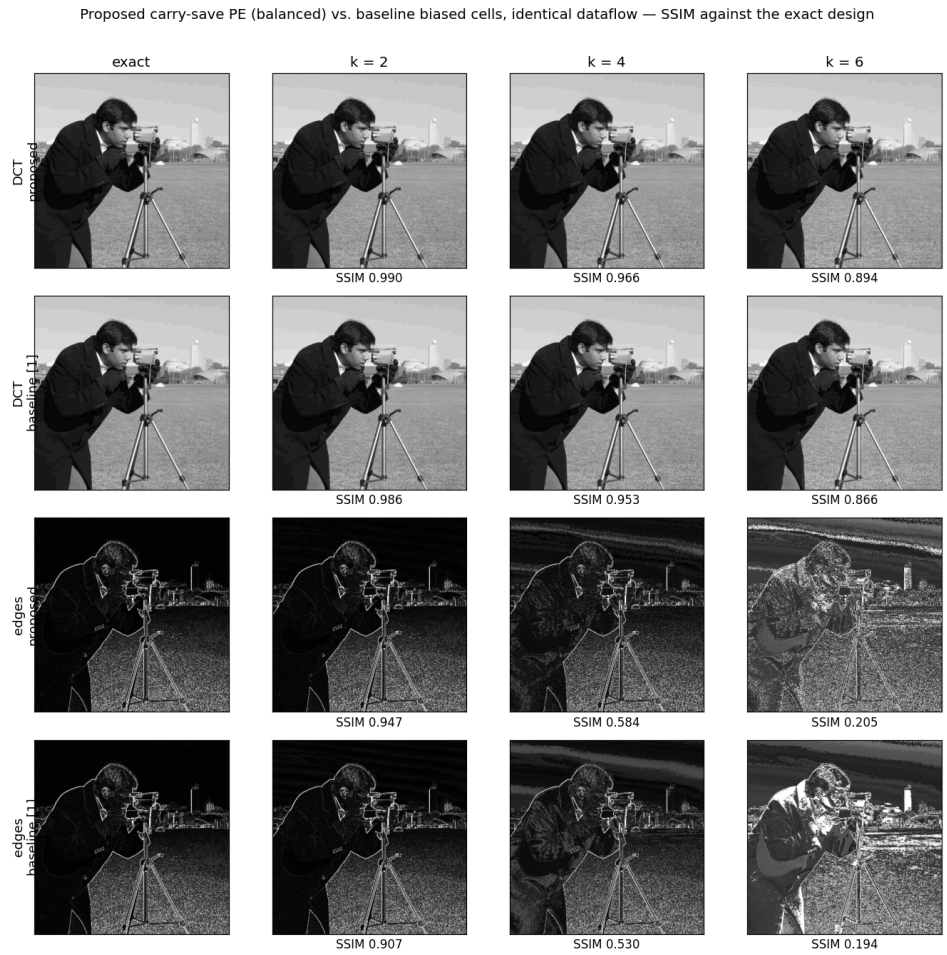


Fig. 6. Application outputs under an identical dataflow, proposed balanced cells versus the baseline's biased cells [1], each annotated with SSIM against the exact design. Rows: DCT compression (proposed, baseline) and Laplacian edge detection (proposed, baseline); columns: the exact PE (bit-identical to true arithmetic) and the approximate PE at $k = 2, 4, 6$. The balanced design is visibly cleaner at every k, most evidently in the edge maps. Test image: `camera` (512×512).

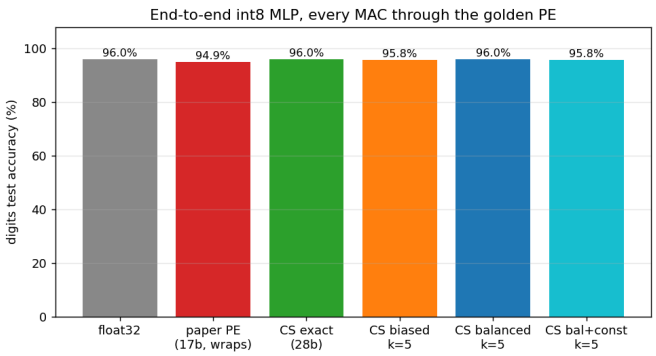


Fig. 7. End-to-end int8 MLP test accuracy with every MAC executed by the golden PE. The resolved 17-bit PE wraps and loses accuracy; the carry-save PE (exact and balanced) matches floating point.

TABLE IV
APPLICATION OUTPUT QUALITY (SSIM VS. THE EXACT DESIGN): PROPOSED BALANCED CELLS VS. THE BASELINE'S BIASED CELLS, IDENTICAL DATAFLOW.

	DCT compression		Laplacian edges	
k	Balanced	Biased [1]	Balanced	Biased [1]
2	**0.990**	0.986	**0.947**	0.907
4	**0.966**	0.953	**0.584**	0.530
6	**0.894**	0.866	**0.205**	0.194

VII. CONCLUSION

We presented a fused carry-save processing element that keeps the baseline's PPC/NPPC cells, retaining their energy benefit, while removing the per-cycle ripple of a resolved accumulator, making the critical-path depth 40 to 45% shorter and independent of accumulator width. An integer Baugh-Wooley identity folds all signed-arithmetic constants to the array boundary, and a structural-zero lemma halves the carry word. We further showed that approximate-cell error in accumulating structures is systematically biased, introduced a complementary cell and a row-parity interleave that reduce the

bias slope $3.3\times$, and cancelled the residual with a free drain-time constant, preserving int8 accuracy where the baseline degrades over long accumulations. All functional claims are exhaustively verified. Future work includes place-and-route power measurement and extension to bit-serial and sparsity-aware dataflows.

REFERENCES

[1] P. Jaswal, L. H. Krishna, and B. Srinivasu, "Energy efficient exact and approximate systolic array architecture for matrix multiplication," arXiv:2509.00778v2 [cs.AR], 2026.

[2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[3] S. Mittal, P. Rajput, and S. Subramoney, "A survey of deep learning on CPUs: opportunities and co-optimizations," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5095–5115, 2022.

[4] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982.

[5] A. J. Abdelmaksoud, S. Agwa, and T. Prodromakis, "DIP: A scalable, energy-efficient systolic array for matrix multiplication acceleration," *IEEE Trans. Circuits Syst. I*, pp. 1–11, 2025.

[6] K. Chen, F. Lombardi, and J. Han, "Matrix multiplication by an inexact systolic array," in *Proc. IEEE/ACM Int. Symp. Nanoscale Architectures (NANOARCH)*, 2015, pp. 151–156.

[7] H. Waris, C. Wang, W. Liu, and F. Lombardi, "AxSA: On the design of high-performance and power-efficient approximate systolic arrays for matrix multiplication," *J. Signal Process. Syst.*, vol. 93, no. 6, 2021, doi:10.1007/s11265-020-01582-7.

[8] H. Waris, C. Wang, W. Liu, and F. Lombardi, "Design and evaluation of a power-efficient approximate systolic array architecture for matrix multiplication," in *Proc. IEEE Int. Workshop Signal Process. Syst. (SiPS)*, 2019, pp. 13–18.

[9] K. Inayat and J. Chung, "Hybrid accumulator factored systolic array for machine learning acceleration," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 7, pp. 881–892, 2022.

[10] M. A. Shafique, K. Inayat, and J. A. Lee, "CSA based radix-4 Gemmini systolic array for machine learning applications," 2023.

[11] H. Genc *et al.*, "Gemmini: enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. DAC*, 2022, pp. 769–774.

[12] C. R. Baugh and B. A. Wooley, "A two's complement parallel array multiplication algorithm," *IEEE Trans. Comput.*, vol. C-22, no. 12, pp. 1045–1047, 1973.

[13] J. Liang, J. Han, and F. Lombardi, "New metrics for the reliability of approximate and probabilistic adders," *IEEE Trans. Comput.*, vol. 62, no. 9, pp. 1760–1771, 2013.

[14] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018, pp. 2704–2713.

[15] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.

[16] P. K. Meher, S. Y. Park, B. K. Mohanty, K. S. Lim, and C. Yeo, "Efficient integer DCT architectures for HEVC," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 24, no. 1, pp. 168–178, 2014.

[17] H. Snopce and A. Aliu, "Latency analysis in the 2-dimensional systolic arrays for matrix multiplication," *Int. J. Comput.*, vol. 15, pp. 1–7, 2021.

[18] C. Wolf, "Yosys open synthesis suite," https://yosyshq.net/yosys/.